

Page Replacement Algorithms

Virtual Memory Management: FIFO vs LRU Simulation

When physical memory is full, the OS must choose which page to swap out to disk to make room for a new one. This guide simulates two common page replacement strategies—First-In, First-Out (FIFO) and Least Recently Used (LRU)—tracking Page Faults and Page Hits.

1. System Configuration Snapshot

We will pass a JSON configuration to our memory management unit (MMU) simulator. It contains a standard reference string representing the sequence of pages requested by a process, and limits physical memory to 3 frames.

```
user@os-sim:~/mmu$ cat paging_config.json
{
  "Reference_String": [7, 0, 1, 2, 0, 3, 0, 4, 2, 3],
  "Frames": 3,
  "Page_Size": "4KB"
}
```

2. FIFO (First-In, First-Out) Algorithm

FIFO replaces the oldest page in memory. It tracks pages in a strict queue. While simple to implement, it ignores how frequently or recently a page is being used, which can lead to high fault rates (and Belady's Anomaly).

```
user@os-sim:~/mmu$ ./pager --algo FIFO --file paging_config.json
```

```
Initializing FIFO Page Replacement Simulator...
```

```
Frames Allocated: 3
```

```
[Req: 7] -> PAGE FAULT! Loaded 7.      Memory: [ 7, -, - ]
```

```
[Req: 0] -> PAGE FAULT! Loaded 0.      Memory: [ 7, 0, - ]
```

```
[Req: 1] -> PAGE FAULT! Loaded 1.      Memory: [ 7, 0, 1 ]
```

```
# Memory is now full. Next fault will trigger an eviction.
```

```
[Req: 2] -> PAGE FAULT! Evicted 7.      Memory: [ 2, 0, 1 ]
```

```
[Req: 0] -> PAGE HIT! 0 is in RAM.  Memory: [ 2, 0, 1 ]
```

```
[Req: 3] -> PAGE FAULT! Evicted 0.      Memory: [ 2, 3, 1 ]
```

```
[Req: 0] -> PAGE FAULT! Evicted 1.      Memory: [ 2, 3, 0 ]
```

```
[Req: 4] -> PAGE FAULT! Evicted 2.      Memory: [ 4, 3, 0 ]
```

```
[Req: 2] -> PAGE FAULT! Evicted 3.      Memory: [ 4, 2, 0 ]
```

```
[Req: 3] -> PAGE FAULT! Evicted 0.      Memory: [ 4, 2, 3 ]
```

```
STATUS: Execution Complete.
```

```
Total Page Faults: 9 | Total Hits: 1 | Hit Ratio: 10%
```

3. LRU (Least Recently Used) Algorithm

LRU assumes that pages used recently will likely be used again soon. When a fault occurs, it searches memory and replaces the page that has not been accessed for the longest period of time.

```
user@os-sim:~/mmu$ ./pager --algo LRU --file paging_config.json
```

```
Initializing LRU Page Replacement Simulator...
```

```
Frames Allocated: 3
```

```
[Req: 7] -> PAGE FAULT! Loaded 7.      Memory: [ 7, -, - ]
```

```
[Req: 0] -> PAGE FAULT! Loaded 0.      Memory: [ 7, 0, - ]
```

```
[Req: 1] -> PAGE FAULT! Loaded 1.      Memory: [ 7, 0, 1 ]
```

```
# Memory full. 7 is the least recently used.
```

```
[Req: 2] -> PAGE FAULT! Evicted 7.      Memory: [ 2, 0, 1 ]
```

```
[Req: 0] -> PAGE HIT! Updated LRU(0). Memory: [ 2, 0, 1 ]
```

```
# Because 0 was just hit, 1 is now the least recently used.
```

```
[Req: 3] -> PAGE FAULT! Evicted 1.      Memory: [ 2, 0, 3 ]
```

```
[Req: 0] -> PAGE HIT! Updated LRU(0). Memory: [ 2, 0, 3 ]
```

```
[Req: 4] -> PAGE FAULT! Evicted 2.      Memory: [ 4, 0, 3 ]
```

```
[Req: 2] -> PAGE FAULT! Evicted 3.      Memory: [ 4, 0, 2 ]
```

```
[Req: 3] -> PAGE FAULT! Evicted 0.      Memory: [ 4, 3, 2 ]
```

```
STATUS: Execution Complete.
```

```
Total Page Faults: 8 | Total Hits: 2 | Hit Ratio: 20%
```

4. Summary & Analysis

A quick comparison of the two algorithm traces and their real-world implications.

```
user@os-sim:~/mmu$ ./pager --analyze
```

```
Running Diagnostics...
```

[FIFO Analysis]

- Pros: Low overhead. Simply uses a circular queue (pointer).
- Cons: Suffered 9 Page Faults. Evicted heavily-used pages (like 0) just because they arrived early.

[LRU Analysis]

- Pros: Better performance (8 Page Faults). Acknowledges temporal locality (saved page 0 from eviction).
- Cons: High overhead. Requires hardware assistance (counters or a doubly-linked list with a hash map) to update the exact usage order on *every single memory access*.